Surname: _____ **First name:** _____

UO: _____

Practical Exam

28 October 2026

Implement `UniqueLinkedCollection<E>` by extending `AbstractCollection<E>`. Use singly linked nodes and preserve insertion order. The collection must not contain `null` elements or duplicates. Do not store the elements in an array or another collection.

Task 1. Representation and insertion [3 points]

Define the node class and implement the methods needed to make the collection fully functional.

Inserting `null` must throw `NullPointerException`. If the collection already contains an equal element, leave it unchanged and return `false`. Otherwise, add the element at the end and return `true`.

Task 2. Constructors [2 points]

Provide the following constructors:

- `UniqueLinkedCollection()`: creates an empty collection.
- `UniqueLinkedCollection(Collection<? extends E> source)`: creates a collection from the elements of `source`, in iteration order. It retains only the first occurrence of each value and throws `NullPointerException` if the source is `null` or contains a `null` element.

Task 3. An iterator with removal [4 points]

Write a private inner class that implements `Iterator<E>` and return a new instance from `iterator()`. Iterators must work independently. Assume that, during a traversal, the collection is modified only through that iterator's `remove()` method.

Operation	Required behaviour
`boolean hasNext()`	Returns `true` if another element is available, without advancing the iterator.
`E next()`	Returns the next element and advances the iterator. Throws `NoSuchElementException` if no elements remain.
`void remove()`	Removes the element returned by the last successful call to `next()`. Throws `IllegalStateException` if `next()` has not yet returned an element, or if that element has already been removed.

Removing an element must not cause the iterator to skip the following one. The collection must still work after any removal, including removal of its only element. Each iterator operation must take O(1) time; `remove()` must not search the chain for the preceding node.

For a collection containing `[A, B, C]`, the following calls produce these results:

Call	Result	Collection afterwards
`next()`	A	[A, B, C]
`remove()`	—	[B, C]
`next()`	B	[B, C]
`next()`	C	[B, C]
`remove()`	—	[B]
`hasNext()`	`false`	[B]

Calling `hasNext()` does not affect whether `remove()` is allowed. A call to `next()` that throws an exception does not change which element can be removed.

Task 4. Main program and supplied tests [1 point]

Write `app.App` to carry out the following checks:

- Create an empty collection, add elements and try adding duplicates. Check the number of elements.
- Construct a collection from repeated values, then change the source and check that the new collection is unaffected.
- Use an iterator to remove the first, a middle and the last element. Remove all elements, then add an element again.
- Check what happens when an iterator has no elements left, when removal is not allowed, and when `null` is inserted.

Print the expected and actual results. Catch expected exceptions so the remaining checks can run.

JUnit tests and the required library are included. Run all supplied tests without modifying them; all must pass. The main program is worth one point, and the collection is assessed in Tasks 1–3. Passing the tests does not replace the main program or the other requirements. No additional public methods are required.